Hadoop Practical 2 — Word Count MapReduce

Problem Statement

Develop a MapReduce program to calculate the frequency of words in a given file.

Theory

What is MapReduce?

MapReduce is a programming model used in Hadoop for processing large amounts of data in parallel.

It works in two phases:

1. Mapper Phase
2. Reducer Phase

Word Count Working

Suppose input file contains:

```
hadoop is big data
hadoop is useful
big data is powerful
```

Mapper Work

Mapper reads one line at a time.

It splits the line into words and emits:

```
(word,1)
```

Example:

```
hadoop → 1
is → 1
big → 1
```

Shuffle and Sort

Hadoop automatically groups same words together.

Example:

```
hadoop → 1,1
is → 1,1,1
big → 1,1
```

Reducer Work

Reducer adds all values of same word.

Example:

```
hadoop → 2
is → 3
big → 2
```

Flow of Word Count

Input File ↓ Mapper ↓ Shuffle and Sort ↓ Reducer ↓ Final Output

---

Mapper Code Explanation

```
import java.io.IOException;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapred.*;
```

Explanation

import java.io.IOException;

Used for handling input-output exceptions.

import org.apache.hadoop.io.*;

Contains Hadoop data types like:

- Text
- IntWritable
- LongWritable

import org.apache.hadoop.mapred.*;

Contains Hadoop MapReduce classes.

```
public class WCMapper extends MapReduceBase
implements Mapper<LongWritable, Text, Text, IntWritable>
```

Explanation

WCMapper

Mapper class name.

MapReduceBase

Base class for Mapper.

Mapper<LongWritable, Text, Text, IntWritable>

Meaning:

Type	Meaning
LongWritable	Input key (line number)
Text	Input value (line text)
Text	Output key (word)
IntWritable	Output value (count)

```
public void map(LongWritable key, Text value,
                OutputCollector<Text, IntWritable> output,
                Reporter rep)
```

Explanation

key

Stores line number.

value

Stores one line from file.

output

Used to send output to reducer.

Reporter

Used to report progress.

```
String line = value.toString();
```

Converts Hadoop Text into normal string.

```
for(String word : line.split(" "))
```

Splits line into words using space.

Example:

```
hadoop is big
```

Becomes:

```
hadoop
is
big
```

```
if(word.length() > 0)
```

Checks word is not empty.

```
output.collect(new Text(word),
               new IntWritable(1));
```

Emits:

```
(word,1)
```

Example:

```
(hadoop,1)
(big,1)
```

Reducer Code Explanation

```
public class WCReducer extends MapReduceBase
implements Reducer<Text, IntWritable, Text, IntWritable>
```

Meaning

Reducer receives:

```
(word,[1,1,1])
```

and returns:

```
(word,total_count)
```

```
public void reduce(Text key, Iterator<IntWritable> values,
                   OutputCollector<Text, IntWritable> output,
                   Reporter rep)
```

Explanation

key

Current word.

values

List of counts.

Example:

```
1,1,1
```

```
int count = 0;
```

Stores total count.

```
while(values.hasNext())
```

Loops through all counts.

```
count += values.next().get();
```

Adds counts.

Example:

```
1 + 1 + 1 = 3
```

```
output.collect(key, new IntWritable(count));
```

Outputs final result.

Example:

```
(hadoop,2)
```

Driver Code Explanation

Driver Role

Driver connects:

- Mapper
- Reducer
- Input Path
- Output Path

and runs Hadoop job.

```
public class WCDriver extends Configured implements Tool
```

Creates Driver class.

```
JobConf conf = new JobConf(WCDriver.class);
```

Creates Hadoop configuration object.

```
FileInputFormat.setInputPaths(conf, new Path(args[0]));
```

Sets input folder.

```
FileOutputFormat.setOutputPath(conf, new Path(args[1]));
```

Sets output folder.

```
conf.setMapperClass(WCMapper.class);
```

Links Mapper class.

```
conf.setReducerClass(WCReducer.class);
```

Links Reducer class.

```
JobClient.runJob(conf);
```

Runs Hadoop job.

Expected Output

```
big 2
data 2
hadoop 2
is 3
powerful 1
useful 1
```

Viva Questions

What is Mapper?

Mapper processes input data and emits key-value pairs.

What is Reducer?

Reducer combines values of same key.

What is Shuffle and Sort?

Hadoop groups same keys together automatically.

Why IntWritable instead of int?

Hadoop uses Writable classes for serialization.

What does Driver do?

Driver configures and runs the Hadoop job.

Hadoop Practical 3 — Student Grade MapReduce

Problem Statement

Develop a MapReduce program to find grades of students.

Theory

Working of Student Grade System

Input:

```
Rahul,95
Amit,82
Sneha,76
```

Mapper emits:

```
(Rahul,95)
(Amit,82)
```

Reducer calculates grade:

```
Rahul → A
Amit → B
```

Flow

Input File ↓ Mapper ↓ Shuffle and Sort ↓ Reducer ↓ Final Grade Output

---

Mapper Code Explanation

```
public class GradeMapper extends Mapper<LongWritable, Text, Text, IntWritable>
```

Meaning

Type	Meaning
LongWritable	Line number
Text	Input line
Text	Student name
IntWritable	Marks

```
String line = value.toString();
```

Converts line into string.

```
String[] fields = line.split(",");
```

Splits line using comma.

Example:

```
Rahul,95
```

Becomes:

```
fields[0] = Rahul
fields[1] = 95
```

```
String student = fields[0];
```

Stores student name.

```
int marks = Integer.parseInt(fields[1]);
```

Converts marks into integer.

```
context.write(new Text(student), new IntWritable(marks));
```

Outputs:

```
(Rahul,95)
```

Reducer Code Explanation

```
public class GradeReducer extends Reducer<Text, IntWritable, Text, Text>
```

Reducer receives:

```
(Rahul,[95])
```

and returns:

```
(Rahul,A)
```

```
int total = 0;
```

Stores total marks.

```
for (IntWritable val : values)
```

Loops through marks.

```
total += val.get();
```

Adds marks.

Grade Logic

```
if (total >= 90)
```

Assigns A grade.

```
else if (total >= 80)
```

Assigns B grade.

```
else if (total >= 70)
```

Assigns C grade.

```
else if (total >= 60)
```

Assigns D grade.

```
else
```

Assigns F grade.

```
context.write(key, new Text(grade));
```

Outputs final grade.

Example:

```
Rahul A
```

Driver Code Explanation

Driver Responsibilities

- Creates Hadoop job
- Connects Mapper and Reducer
- Sets input/output paths
- Runs program

```
Configuration conf = new Configuration();
```

Creates Hadoop configuration.

```
Job job = Job.getInstance(conf, "Student Grades");
```

Creates job object.

```
job.setJarByClass(GradeDriver.class);
```

Specifies Driver class.

```
job.setMapperClass(GradeMapper.class);
```

Links Mapper.

```
job.setReducerClass(GradeReducer.class);
```

Links Reducer.

```
FileInputFormat.addInputPath(job, new Path(args[0]));
```

Sets input path.

```
FileOutputFormat.setOutputPath(job, new Path(args[1]));
```

Sets output path.

```
System.exit(job.waitForCompletion(true) ? 0 : 1);
```

Runs job and exits.

Expected Output

```
Amit B
Priya D
Rahul A
Rohan F
Sneha C
```

Viva Questions

What does Mapper do?

Reads student data and emits student name with marks.

What does Reducer do?

Calculates total marks and assigns grade.

What is the role of Driver?

Configures and runs Hadoop job.

Why use Hadoop?

To process large data efficiently using distributed computing.

What is distributed processing?

Processing data across multiple systems together.